

Deployment Guide

Cybersecurity Temporal Threat Correlation — run the trained network as an advisory worker

The deployment half of the pipeline: take the JNeopallium network the trainer produced, wire it to your event sources and output sink, and run it as a continuous, shadow-to-advisory security worker — configured exactly as the architecture intends.

Document	Production Deployment Guide
Product	Jneopallium Cybersecurity Module (Demo 06)
Consumes	The deployable network from the Training Guide
Safety mode	ADVISORY (recommend-only)
Platforms	Windows (PowerShell) and Linux/macOS (bash)
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	23 June 2026
License	BSD 3-Clause

Contents

1. What you will achieve
2. Prerequisites — the trained package and runtime
3. The launch architecture
4. The deployable package (what training produced)
5. Point the worker at the generated neuron-net structure
6. Package the deployment JAR
7. Runtime configuration: the IContext property reference
8. Configure event sources and streaming input
9. Wire the output aggregator and launch the worker
10. Promote from shadow mode to operator-visible advisory
11. Monitoring in production
12. Safety, rollback and packaging
13. Troubleshooting
14. Appendix — deployment cheat-sheet

1 What you will achieve

By the end of this guide you will have taken the **deployable network produced by the Training Guide** — a directory of JNeopallium layer and neuron configuration with real runtime classes and embedded trained weights — and run it as a continuous **advisory worker** that recommends, but never autonomously enforces, security responses.

Safety ceiling for the whole guide

Everything below operates in ADVISORY mode. The worker may recommend investigation or quarantine candidates, but it must not isolate hosts or block traffic without a separate safety case, approval workflow, and rollback path. The trained network already marks the response layer ADVISORY and the hard gate as fixed configuration.

2 Prerequisites — the trained package and runtime

- **A trained network** from the Training Guide — the artifact directory containing `model-descriptor.json`, `layer-0.json` ... `result-layer.json`, and `trained-temporal-threat-model.json`.
- **Java 17+** to run the worker, and the built worker artifact (`mvn clean install`).
- **Your telemetry source** — a Kafka-style event bus (or any feed you can adapt to typed signals).
- **An output destination** — a JSONL sink for shadow mode, or your SIEM / Kafka topic / webhook for operator-visible advisory.

Training first

If you do not yet have a trained network, run the companion Training Guide. Deployment consumes its output directory directly — there is no manual translation step between training and deployment.

3 The launch architecture

A production deployment uses the worker's real entry point — the architecture intends you to supply your own JAR, the trained network, configuration, and event sources. The worker is started by one entry point with four arguments:

```
java com.rakovpublic.jneuropallium.worker.application.Entry \  
  <mode> <model-jar-url> <context-class> <context-json-or-path>
```

Argument	Meaning	Example
mode	Deployment topology	local, http, grpc-client, grpc-master
model-jar-url	URL of your deployment JAR	file:///opt/jneo/cyber-model.jar
context-class	An IContext implementation	...runtime.DemoJsonContext
context-json-or-path	Inline JSON or a path to a JSON config file	/opt/jneo/cyber-context.json

Internally, Entry hands the four arguments to the Runner, which: (1) deserialises the context JSON into your IContext class with Jackson; (2) loads the JAR into an isolated class loader (JarClassLoaderService); (3) selects an application by mode; and (4) starts it. The mode maps directly onto the deployment topologies from the architecture:

Mode	Application	Topology
local	LocalApplication	Single JVM — pilots, edge sites
http	HttpClusterApplication	Distributed worker nodes over HTTP
grpc-master / grpc-client	GRPCServerApplication / GRPCClient	gRPC cluster; FPGA-capable

The four production ingredients

A deployment is exactly four things: (1) the **trained network** (from training), (2) the **IContext configuration**, (3) a **deployment JAR** (worker security classes plus your event source and sink), and (4) your **event sources and output sink**. The next sections wire each one.

4 The deployable package (what training produced)

The trainer does not emit an abstract model that you must translate — it emits the **actual JNeopallium network**. The package is a directory of ten files; the five layer files plus the descriptor are what the worker boots from:

Artifact	Role at deploy time
model-descriptor.json	Whole-network map: 5 layers, 8 neurons, frequency map, runtime classes
layer-0.json	Multi-source event input boundary
layer-1-fast-evidence.json	Fast evidence receptors (4 neurons)
layer-2-temporal-correlation.json	Trained correlator — embedded weights, gates, baseline policy
layer-3-response-planning.json	Advisory response bands + the fixed hard safety gate
result-layer.json	Security-advisory result output
trained-temporal-threat-model.json	Compact trained model the correlation layer references
trained-model-update.json / quantitative-summary.json / source-mapping.json	Provenance, metrics, and source mapping

Total network: **5 layers, 8 real neurons, 34 trainable weights, 1 bias**. Every neuron names a concrete class from the worker's security package — so deployment is a matter of pointing the worker at this directory, not re-implementing anything.

5 Point the worker at the generated neuron-net structure

The neuron-net structure — the architecture's "neuron-network structure file" — is **generated by training**, not hand-written. Point `configuration.input.layermeta` at the directory holding the five layer files and the worker boots the network directly. Each file is a real layer of real neurons:

Layer file	Layer	Real neuron classes
layer-0.json	Input	(no neuron objects; canonical inputs only)
layer-1-fast-evidence.json	Fast evidence	NetworkFlowNeuron, SignaturePatternNeuron, ProcessBehaviourNeuron, EntityBehaviourBaselineNeuron
layer-2-temporal-correlation.json	Correlation	TemporalThreatCorrelationNeuron
layer-3-response-planning.json	Planning + gate	ResponsePlanningNeuron, ResponseGateNeuron
result-layer.json	Result	ResponsePlanningNeuron (result)

The correlation layer embeds everything the model learned, ready to run:

```
"dendrites": {
  "weights": { "technique_command_and_control": 0.428,
               "network_receptor_score": 0.424,
               "maintenance_ratio": -0.452, ... },
  "bias": 2.1597, "decisionThreshold": 0.2 },
"baselineAdaptation": {
  "freezeWhenPosteriorAtLeast": 0.3,
  "freezeWhenSignatureConfidenceAtLeast": 0.8,
  "trustedBenignOnly": true }
```

No translation step

Because the structure is generated with the trained weights already inside, what you deploy is exactly what was trained and tested. The decision threshold (0.2), the five sequence gates, the response bands, and the baseline-freeze thresholds all travel inside the layer files — nothing is re-entered by hand.

6 Package the deployment JAR

The neuron, processor, and signal classes the structure references **already ship in the worker's security module**, so you do not re-implement them:

- Neurons — `com.rakovpublic.jneuropallium.worker.net.neuron.impl.security.*` (TemporalThreatCorrelationNeuron, ResponseGateNeuron, ...).
- Processors — `...worker.signalprocessor.impl.security.*` (AnomalyHypothesisProcessor, HypothesisResponseProcessor, QuarantineGateProcessor, ...).
- Signals — `...worker.net.signals.impl.security.*` (AnomalyScoreSignal, ThreatHypothesisSignal, QuarantineRequestSignal, ...).

Your deployment JAR therefore needs to provide only the two integration pieces that are specific to your environment: a custom **event source** (`IInitInput`, e.g. a Kafka reader) and a custom **output sink** (`IOutputAggregator`, e.g. a SIEM forwarder). Build it as a Maven module that depends on the worker artifact:

```
mvn -pl cyber-deploy clean package
# -> target/cyber-model-1.0.jar (your event source + sink + worker on the classpath)
# passed to Entry as: file:///opt/jneo/cyber-model.jar
```

Fail-fast class check

List the runtime classes your structure references in `configuration.neuronnet.classes` (the trainer records them in `model-descriptor.json` under `generatedFrom.sourceRuntimeClasses`). At startup the worker verifies each is present and aborts with a clear error if any is missing.

7 Runtime configuration: the `IContext` property reference

Configuration reaches the worker through an `IContext`. The bundled `DemoJsonContext` reads a JSON document holding a flat properties map; the alternative `Context` class reads a classic `.properties` file. Both expose the same keys via `getProperty(...)`. The properties the worker reads, with the values that define a **deployed advisory (inference) service**:

Property	Purpose / advisory value
<code>configuration.input.layermeta</code>	Directory of the generated per-layer structure files
<code>configuration.neuronnet.classes</code>	CSV of runtime classes that must exist in the JAR (validated at startup)
<code>configuration.input.inputs</code>	Event sources — an <code>InputArray</code> of <code>IInitInput</code> bindings
<code>configuration.outputAggregator</code>	<code>IOutputAggregator</code> class that receives every advisory
<code>configuration.storage.json</code>	<code>IStorage</code> definition (filesystem state for neuron / history dumps)
<code>configuration.processing.frequency.map</code>	Per-signal {epoch, loop} cadence (from the descriptor)
<code>configuration.slowfast.ratio</code>	Fast-to-slow loop ratio (N)
<code>configuration.history.slow.runs / .fast.runs</code>	Signal-history depth for the slow / fast loop
<code>configuration.isteacherstudying</code>	<code>`false`</code> for a deployed advisory worker; true only to teach/learn
<code>configuration.infiniteRun</code>	<code>`true`</code> for a continuous streaming service
<code>configuration.maxRun</code>	Run count when not infinite (batch replay)
<code>configuration.runoncein</code>	Milliseconds to sleep between cycles (poll cadence)
<code>configuration.discriminatorsAmount</code>	<code>`0`</code> for a plain advisory worker
<code>worker.threads.amount</code>	Worker threads per layer

Inference vs. teaching mode

Set `configuration.isteacherstudying=false` with `discriminatorsAmount=0` and `infiniteRun=true`. This puts the worker into its continuous advisory loop: pull signals from the event sources → run the trained net → write advisories through the aggregator → repeat. Teaching mode (`true`) is for training, not for a live detector.

8 Configure event sources and streaming input

Event sources are where production telemetry enters the net. `configuration.input.inputs` is an `InputArray` of one or more sources; each binds an `IInitInput` (the source) to an `InputInitStrategy` (how its signals are placed into the first layer):

```
{
  "inputData": [
    {
      "iInputSource": {
        "clazz": "com.acme.cyber.KafkaSecurityEventSource",
        "initInput": { "name": "cyber-stream", "epoch": 1, "loop": 1,
                      "topics": "auth,proc,dns,flow,intel" }
      },
      "mandatory": true,
      "initStrategy": {
        "clazz": "...signals.OneToAllFirstLayerInputStrategy",
        "iNeuronNetInput": {}
      },
      "amountOfRuns": 0
    }
  ]
}
```

An event source implements the `IInitInput` contract — four methods: `readSignals()` returns the typed signals for this cycle; `getName()`; `getDesiredResults()` (training only — empty for a live detector); and `getDefaultProcessingFrequency()`.

Plugging in a real Kafka event source

To consume your live telemetry, implement an `IInitInput` that polls your Kafka topic(s) and converts each record into the typed security signals the network consumes — `PacketSignal`, `SyscallSignal`, `LogEventSignal` at the receptors, feeding `SignatureMatchSignal` and `AnomalyScoreSignal` into the correlator. Reference its class as the source `clazz`. Keep each event's **event-time** field so delayed, replayed, or out-of-order telemetry still correlates by event time, exactly as the architecture requires.

Continuous streaming vs. batch replay

For a long-running service set `configuration.infiniteRun=true` and `configuration.runoncein=<ms>` as the poll interval; the worker loops, pulling from the event source and emitting advisories continuously. For a bounded replay (for example, re-running an incident), set `infiniteRun=false` and `configuration.maxRun=<cycles>`.

Per-signal cadence (typed timescales)

`configuration.processing.frequency.map` assigns each signal its natural timescale via `{epoch, loop}`. The trainer records the recommended cadence in the descriptor's `signalFrequencyMap` — fast receptors every cycle, hypotheses and quarantine requests every second cycle, incident reports every fifth:

```
{
  "...security.PacketSignal":      { "epoch": "1", "loop": "1" },
  "...security.SyscallSignal":    { "epoch": "1", "loop": "1" },
  "...security.AnomalyScoreSignal": { "epoch": "1", "loop": "1" },
  "...security.ThreatHypothesisSignal": { "epoch": "1", "loop": "2" },
  "...security.IncidentReportSignal": { "epoch": "1", "loop": "5" }
}
```

9 Wire the output aggregator and launch the worker

`configuration.outputAggregator` names an `IOutputAggregator`. Its single method — `save(results, timestamp, run, context)` — is called every cycle with the advisories the net produced. The bundled `JsonlResultAggregator` writes advisory JSONL, which is ideal as the **shadow-mode audit sink**. For production, implement an aggregator that forwards advisories to your SIEM, a Kafka topic, or a webhook — preserving the full evidence lineage (entity, event-time range, contributing sources, posterior, response band, baseline-freeze state, model id and checksum).

A complete production context pulls it all together — note `configuration.input.layermeta` pointing at the generated layer directory:

```
{
  "properties": {
    "configuration.input.layermeta": "/opt/jneo/cyber/layers",
    "configuration.neuronnet.classes": "...security.TemporalThreatCorrelationNeuron,...security.ResponsePlanningNeuron,...security.ResponseGateNeuron",
    "configuration.input.inputs": "{ ...InputArray from section 8... }",
    "configuration.processing.frequency.map": "{ ...frequency map... }",
    "configuration.outputAggregator": "com.acme.cyber.SiemAdvisorySink",
    "configuration.storage.json": "{ \"storageClass\": \"...DemoFileStorage\", \"storage\": { \"rootPath\": \".../opt/jneo/cyber/state\" } }",
    "configuration.slowfast.ratio": "10",
    "configuration.history.slow.runs": "8",
    "configuration.history.fast.runs": "64",
    "configuration.isteacherstudying": "false",
    "configuration.discriminatorsAmount": "0",
    "configuration.infiniteRun": "true",
    "configuration.runoncein": "1000",
    "configuration.maxRun": "0",
    "worker.threads.amount": "4"
  }
}
```

Launch the worker, putting the worker artifact and its dependencies on the classpath and passing your deployment JAR by URL (use `;` as the classpath separator on Windows, `:` on Linux/macOS):

```
java -cp "worker-1.0.jar:libs/*" \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local \
  file:///opt/jneo/cyber-model.jar \
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
```


Start in shadow mode

Point `configuration.outputAggregator` at an audit-only sink first (the JSONL aggregator is perfect), with no operator notification. The worker streams advisories you can compare against incident tickets before promoting to operator-visible advisory — the subject of the next section.

10 Promote from shadow mode to operator-visible advisory

Promote the network gradually. Never jump straight to operator-visible output, and never enable enforcement without a separate approved design.

1. **Deploy adapters** that convert real telemetry into the typed security signals.
2. **Load the network** by pointing `configuration.input.layermeta` at the generated layer directory.
3. **Start in shadow mode:** write advisories to an audit topic or JSONL sink without notifying operators.
4. **Compare** advisories against incident tickets, red-team logs, and sampled benign activity.
5. **Promote to operator-visible advisory** only after false-positive budgets and latency targets are met.
6. **Keep active enforcement disabled** unless a separate response-control design is approved.

A real Kafka bridge can replace the demo input. Keep the same typed event contract and event-time fields so delayed, replayed, or out-of-order telemetry is still correlated by event time rather than by processing time.

11 Monitoring in production

Track these metrics continuously:

- Events processed per source per minute; dropped, delayed, and out-of-order events.
- Advisory count per host group and tenant; false-positive review rate; confirmed true-positive rate.
- Mean time to detection and mean time to operator acknowledgement.
- Baseline updates accepted, frozen, and rejected; maintenance-window suppression rate.
- Model version and threshold used for every advisory.

Every advisory must preserve its evidence lineage: `entity_id`, the `event_tick` range, the contributing sources and techniques, the posterior, the response band, `baselineFrozen`, the `modelId`, and the model checksum (the `trainingChecksum` from `trained-model-update.json`).

12 Safety, rollback and packaging

Packaging

A deployment package is the generated artifact directory plus provenance:

- The generated layer files and model artifacts (`model-descriptor.json`, `layer-0.json` ... `result-layer.json`, `trained-temporal-threat-model.json`, `trained-model-update.json`).

- Git commit SHA, training command, manifest checksum, the `trainingChecksum`, generated artifact checksums, the advisory-deployment approval record, and a rollback package identifier.

Safety and rollback rules

- Hard safety gates are fixed configuration in `layer-3-response-planning.json`, never learned model weights.
- Critical-asset allow-lists and maintenance windows reduce urgency but never erase evidence.
- Baseline adaptation freezes when the posterior reaches 0.3 or signature confidence reaches 0.8 — encoded in the layer's `baselineAdaptation` block.
- Keep the previous network package available for immediate rollback.
- Roll back if false positives exceed budget, source coverage drops, or parsing errors create systematic blind spots.

Honest limitation

The checked-in reference network is trained on a synthetic, deterministic corpus. It is excellent for repeatable pipeline evidence and safety-gate regression, but it does not establish real-world detection quality. Production claims require retraining on external multi-source datasets (see the Training Guide) and validation on representative enterprise telemetry.

13 Troubleshooting

Symptom	Likely cause and fix
Cannot find class ... in provided jar	A class in <code>configuration.neuronnet.classes</code> is missing — rebuild the deployment JAR
Worker starts but emits nothing	Check the event source <code>clazz</code> and that <code>readSignals()</code> returns typed signals
Layer not loaded / structure error	Confirm <code>configuration.input.layermeta</code> points at the generated layer directory
Worker exits immediately	For a live detector set <code>infiniteRun=true</code> ; <code>maxRun</code> only applies to bounded replay
Advisories emit a blocking action	Verify the response layer is <code>ADVISORY</code> ; this is a regression to report
Wrong threshold or weights at runtime	You deployed a stale layer directory; redeploy the latest trained package
Cyrillic / encoding issues in output	Ensure the aggregator writes UTF-8

14 Appendix — deployment cheat-sheet

Launch a production advisory worker

```
java -cp "worker-1.0.jar:libs/*" \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local file:///opt/jneo/cyber-model.jar \
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
  /opt/jneo/cyber-context.json
```

The advisory-worker context essentials

```
configuration.input.layermeta    = <generated layer directory>
configuration.isteacherstudying = false
configuration.discriminatorsAmount = 0
configuration.infiniteRun       = true
configuration.runoncein         = 1000
configuration.outputAggregator  = <your IOutputAggregator>
```

Jneopallium Cybersecurity Module · Demo 06 · Deployment Guide. Consumes the network produced by the Training Guide. Safety mode: ADVISORY. Active enforcement requires a separate approved safety case. License: BSD 3-Clause.